PROMPT 5 — Lógica Fake en el Navegador (Web)

> Documento-prompt para una IA implementadora. Debe **crear el código JS** de la lógica fake de la web y entregarlo funcionando + pruebas automáticas en verde.
>
> **Ante cualquier duda o falta de información: PÁRATE y pregunta al responsable del proyecto ANTES de escribir o modificar nada.** No asumas ni inventes.

---

1. Rol y tarea

Eres un ingeniero frontend (JavaScript vanilla, sin librerías en runtime).

Tarea: implementar la **lógica fake del navegador** (sección 3, marcada `(new)`) que recupera las mediciones del servidor y las deja listas para que la UX (Prompt 6) las pinte.

2. Contexto del proyecto

- El servidor REST responde **`GET /mediciones`** con `{"mediciones": [Medicion...]}` (Prompt 3).
- Cada `Medicion`: `{ id: N, uuid: Text, major: N, minor: N, tx_power: N, nombre_emisora: Text, fecha_hora: Text }`.
- El valor de la medición es **`minor`** (p.ej. 1234).
- Regla de capas: `LogicaFake` hace la petición HTTP **directamente con `fetch`** (no hay cliente REST aparte). Y **no pinta**: devuelve datos limpios para la UX.

3. Diseño a implementar

3.1 Objeto NUEVO `LogicaFake`

```
      ------------------- LogicaFake (NEW) -------------------
      |
      | URL_MEDICIONES: Text      (const = "/mediciones")
      |
      |
      --> recuperarMedicion() -->              // (new) nombre singular del
dibujo,
mediciones: [Medicion] <--                     //       devuelve la LISTA
completa
      |
      ------------------------------------------------------------------
```

Método	Firma	Comportamiento
`recuperarMedicion()`	`--x --> mediciones: [Medicion]`	async. Hace `fetch` con `"GET"` a `URL_MEDICIONES`. Si `res.ok` → parsea el JSON, toma `datos.mediciones` (si no es array → rechaza con mensaje en español) y devuelve el array. Si no `res.ok` → rechaza con el `{error}` del cuerpo (o mensaje

Método	Firma	Comportamiento
		en español si no viene). Cada Medicion se "limpia": `minor`, `major`, `tx_power` a `Number` (`uuid`, `nombre_emisora`, `fecha_hora` a cadena).

> `URL_MEDICIONES`: con **XAMPP**, apunta a la URL real del recurso según dónde cuelgue la web en `htdocs` (p.ej. `"/mediciones"` si `backend/public` es la raíz de Apache, o `"http://localhost/<carpeta>/mediciones"` si va en subcarpeta).

4. Ficheros a entregar

En `web/`:

- `js/logica_fake.js` — objeto `LogicaFake` (con su `fetch`).
- (`index.html`, `css/estilo.css`, `js/ux.js` los genera el Prompt 6; aquí solo `LogicaFake`.)

Carga en el navegador: `logica_fake.js` → `ux.js`.

5. Requisitos de código y de comentarios (OBLIGATORIOS)

1. Comentarios **en español**, simples.
   - Cabecera de cada fichero: qué hace y a qué objetivo sirve.
   - Cada función con 2–4 líneas antes de la firma: `PROPOSITO` / `PARAM.` / `RETORNA`.
2. Logs con `console.log('>>>>', ...)` en cada paso (paralelo a los `Log.d` de Android).

6. Pruebas automáticas (OBLIGATORIAS)

6.1 `web/tests/logica_fake.test.js` (Jest, con `fetch` mockeado; `package.json` con `jest ^29`)

1. **`recuperarMedicion` resuelve según el servidor**: mock con `200 {"mediciones":[{ "minor":"1234", "uuid":"EPSG-GTI-PROY-3A", "major":"2816", "tx_power":"4", "nombre_emisora":"Minerva_ELBACON", "fecha_hora":"2026-09-28 09:00:00" }]}` → array de 1 elemento, `minor === 1234` (número) y `nombre_emisora === "Minerva_ELBACON"`.
2. **Array vacío** si `{"mediciones":[]}` → `[]`.
3. **Rechaza si el servidor falla**: mock `{ok:false, status:500, json:()=>({error:"error interno"})}` → mensaje con "error".
4. **`recuperarMedicion` llama a `fetch` con `"GET"` y `/mediciones`**: espiar `global.fetch` y comprobar método y ruta.

6.2 Integración real

BBDD con datos importada con phpMyAdmin (Prompt 1) + Apache de XAMPP sirviendo `backend/public` (Prompt 3) → abrir `http://localhost/<carpeta>/` y ver la lista (Prompt 6).

6.3 Ejecución

```
cd web
npm install
npm test
```

Todos en verde.

7. Verificación y entrega

- [ ] `LogicaFake.recuperarMedicion()` (singular) hace `fetch` a `/mediciones` y devuelve `[Medicion]` limpio.
- [ ] La lógica fake no pinta; devuelve datos limpios.
- [ ] Comentarios en español. `npm test` en verde. Resumen final: ficheros y comandos.